

MANUAL BOOK

CloudVault

Smart Object Storage Manager

TUGAS BESAR – CLOUD COMPUTING

Oleh : Muhammad Yusuf Anwar

NIM : 32602500068

Kelas : RPL LP MAARIF

Program Studi Teknik Informatika

Fakultas Teknologi Industri

Universitas Islam Sultan Agung

Daftar Isi

- Pendahuluan
- Tujuan Aplikasi
- Kebutuhan Sistem
- Instalasi
- Menjalankan MiniStack
- Menjalankan StackPort
- Menjalankan JupyterLab
- Panduan Penggunaan
- Pengujian
- Kesimpulan

BAB I - Pendahuluan

CloudVault adalah aplikasi sederhana untuk mengelola Object Storage menggunakan Amazon S3 API melalui MiniStack sebagai emulator AWS lokal.

BAB II - Tujuan Aplikasi

- Memahami konsep Object Storage
- Menggunakan Boto3
- Mengelola bucket dan object pada S3.

BAB III - Kebutuhan Sistem

Hardware: Laptop/PC, RAM minimal 4 GB.

Software: Windows 10/11, Python 3.13, JupyterLab, boto3, MiniStack, StackPort.

BAB IV - Instalasi

1. Install Python
2. Install JupyterLab (pip install jupyterlab)
3. Install boto3 (pip install boto3)
4. Jalankan JupyterLab.

BAB V - Menjalankan MiniStack

Jalankan MiniStack dan pastikan endpoint `http://127.0.0.1:4566` aktif.

```
C:\Windows\system32\cmd.exe - ministack
Required-by:
C:\Users\User>pip show stackport
Name: stackport
Version: 0.3.0
Summary: Universal AWS resource browser for local emulators
Home-page: https://github.com/DaviReisVieira/stackport
Author: Davi Reis Vieira
Author-email:
License: MIT
Location: C:\Users\User\AppData\Local\Programs\Python\Python313\Lib\site-packages
Requires: boto3, click, fastapi, python-multipart, uvicorn, websockets
Required-by:
C:\Users\User>ministack
2026-07-19 14:30:12 INFO [ministack]
MO
MiniStack
Local AWS Service Emulator - Port 4566
Services: S3, SQS, SNS, DynamoDB, Lambda, IAM, STS, SecretsManager, CloudWatch Logs,
          SSM, EventBridge, Kinesis, CloudWatch, SES, SES v2, ACM, WAF v2, Step Functions,
          ECS, RDS, ElastiCache, Glue, Athena, API Gateway, Firehose, Route53,
          Cognito, EC2, EMR, EBS, EFS, ALB/ELBv2, CloudFormation, KMS, ECR, CloudFront,
          AppSync, Cloud Map, S3 Files, RDS Data API, CodeBuild, AppConfig, Transfer, EKS,
          Inspector2, IoT Core
2026-07-19 14:30:12 INFO [ministack] Worker thread pool: 64 threads
2026-07-19 14:30:12 INFO [transfer] SFTP: disabled (asyncssh missing or SFTP_ENABLED=0)
2026-07-19 14:30:12 WARNING [ministack] Scheduler startup failed: name 'method' is not defined
2026-07-19 14:30:12 INFO [ministack] Ready - 74 services available on port 4566.
[2026-07-19 14:30:12 +0700] [2540] [INFO] Running on http://0.0.0.0:4566 (CTRL + C to quit)
2026-07-19 14:30:12 INFO [hypercorn.error] Running on http://0.0.0.0:4566 (CTRL + C to quit)
2026-07-19 14:32:15 WARNING [kms] cryptography package not installed; KMS Sign/Verify will return errors. Install with: pip install cryptography
2026-07-19 14:56:31 INFO [s3] S3 bucket created: cloudvault-storage
```

BAB VI - Menjalankan StackPort

Tambahkan endpoint MiniStack, gunakan region `us-east-1` dan Static Credentials (`test/test`).

StackPort

AWS Resource Browser

Dashboard

Resources

Settings

About

MinStack us-east-1

Local (127.0.0.1:4566)

Writes enabled

us-east-1

StackPort

Settings

Manage StackPort configuration and preferences

Endpoints

Endpoint Configuration

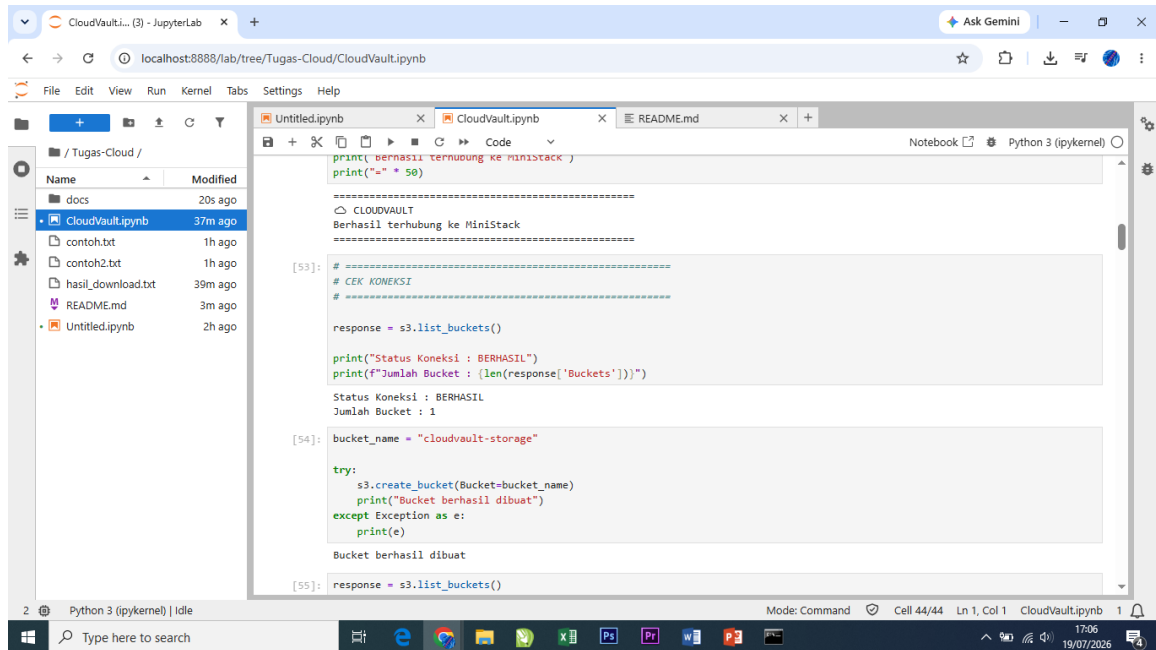
Manage AWS endpoints and local emulators. Switch between endpoints to browse different environments.

+ Add Endpoint

Name	URL / Type	Region	Auth	Health	Source	Actions
default	Real AWS	us-east-1	Default	Unhealthy	Environment	
MiniStack	http://127.0.0.1:4566	us-east-1	Keys	Healthy	User	

BAB VII - Menjalankan CloudVault

Buka CloudVault.ipynb kemudian Run All Cells.



```
print('Berhasil terhubung ke MinioStack')
print("=" * 50)

=====
CLOUDVAULT
Berhasil terhubung ke MinioStack
=====

[53]: # =====
# CEK KONEKSI
# =====

response = s3.list_buckets()

print("Status Koneksi : BERHASIL")
print(f"Jumlah Bucket : {len(response['Buckets'])}")

Status Koneksi : BERHASIL
Jumlah Bucket : 1

[54]: bucket_name = "cloudvault-storage"

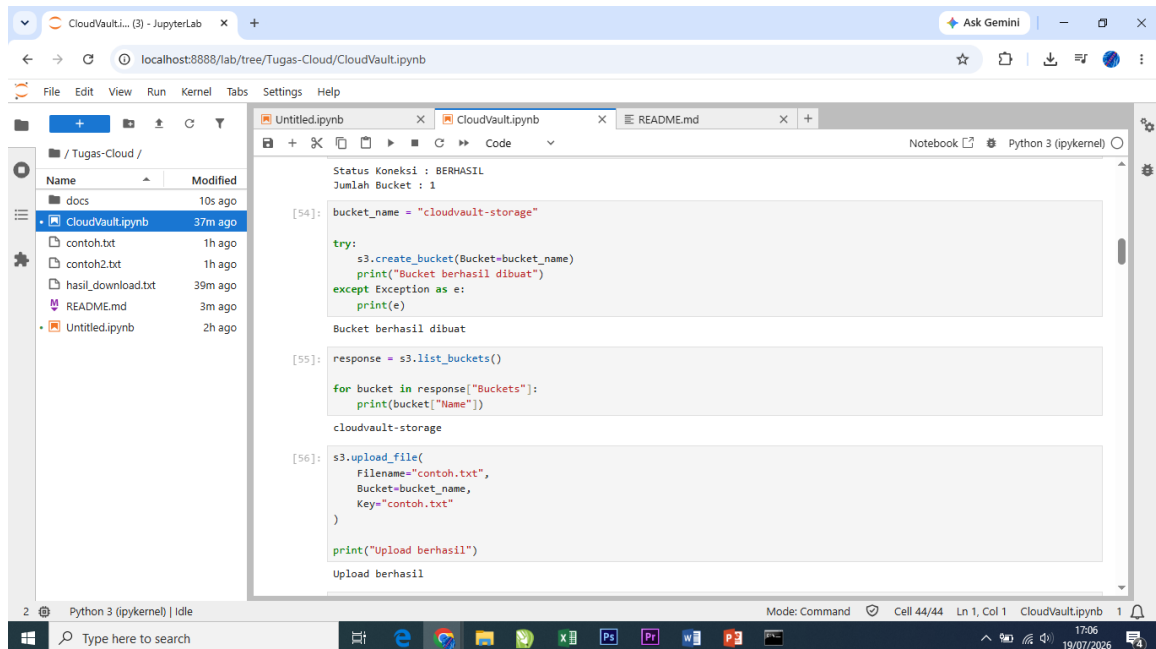
try:
    s3.create_bucket(Bucket=bucket_name)
    print("Bucket berhasil dibuat")
except Exception as e:
    print(e)

Bucket berhasil dibuat

[55]: response = s3.list_buckets()
```

BAB VIII - Panduan Penggunaan

• Create Bucket



```
Status Koneksi : BERHASIL
Jumlah Bucket : 1

[54]: bucket_name = "cloudvault-storage"

try:
    s3.create_bucket(Bucket=bucket_name)
    print("Bucket berhasil dibuat")
except Exception as e:
    print(e)

Bucket berhasil dibuat

[55]: response = s3.list_buckets()

for bucket in response["Buckets"]:
    print(bucket["Name"])

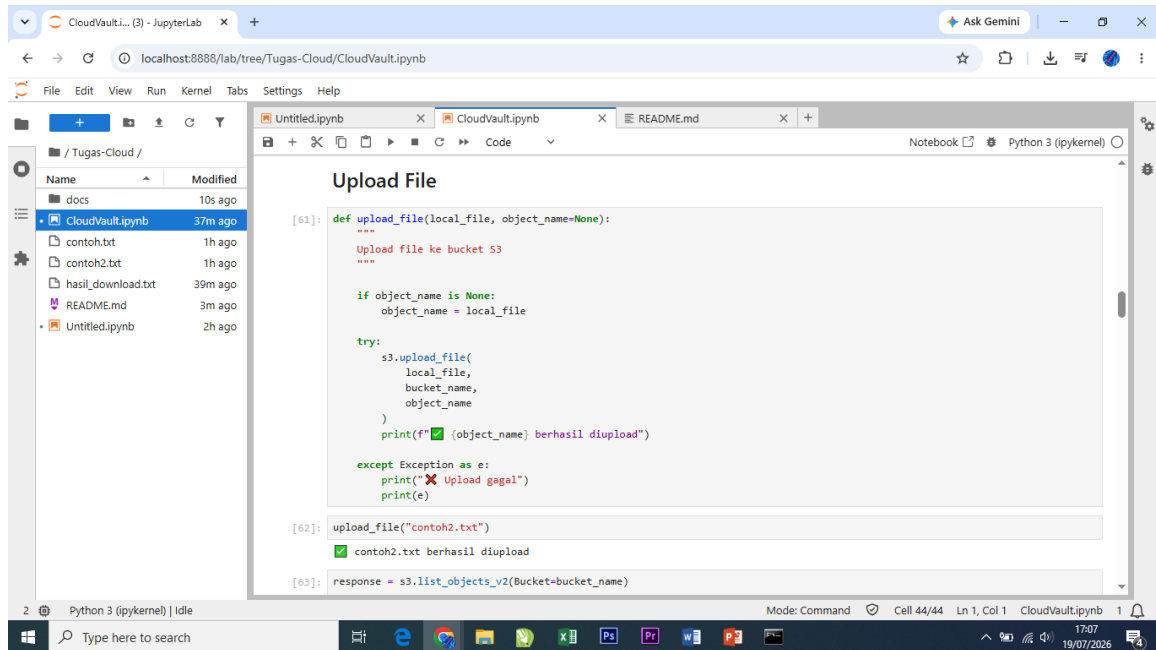
cloudvault-storage

[56]: s3.upload_file(
    Filename="contoh.txt",
    Bucket=bucket_name,
    Key="contoh.txt"
)

print("Upload berhasil")

Upload berhasil
```

• Upload File



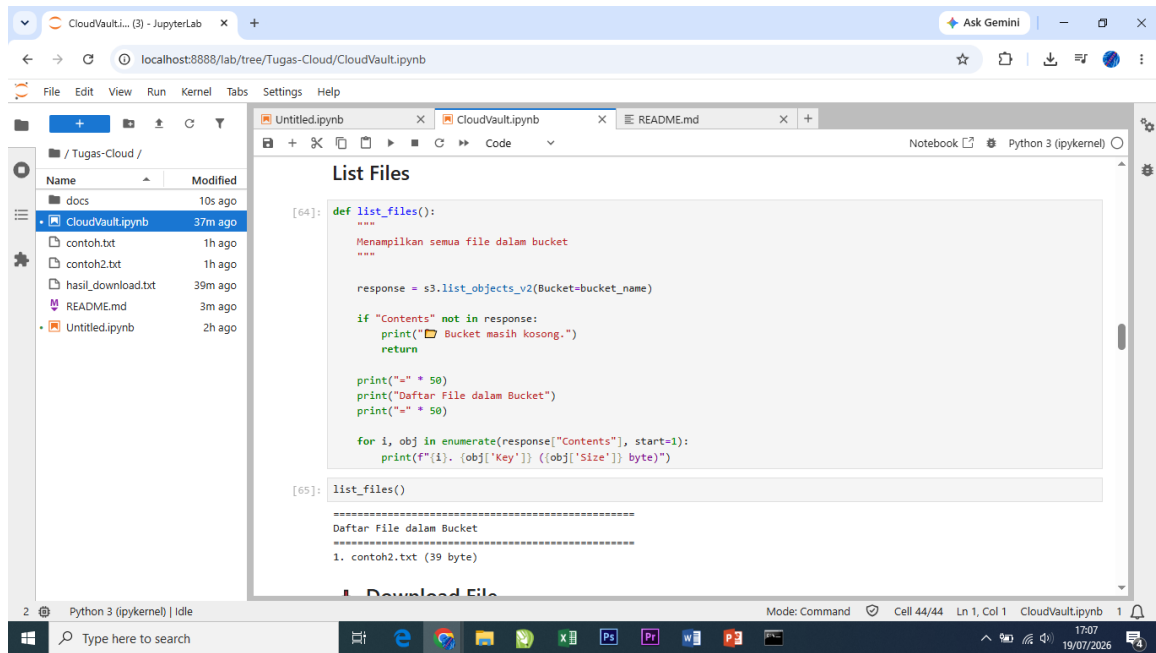
The screenshot shows a JupyterLab interface with a file browser on the left and a code editor on the right. The file browser displays a directory structure with files like `docs`, `CloudVault.ipynb`, `contoh.txt`, `contoh2.txt`, `hasil_download.txt`, `README.md`, and `Untitled.ipynb`. The code editor contains a Python function `upload_file` that takes a local file path and an object name as arguments. It uses the `s3.upload_file` method to upload the file to an S3 bucket. The function also includes error handling and prints a success message. The notebook shows the execution of the function with `upload_file("contoh2.txt")`, resulting in a success message: `contoh2.txt berhasil diupload`. The status bar at the bottom indicates the notebook is running on Python 3 (ipykernel) in Command mode.

```
[61]: def upload_file(local_file, object_name=None):
      """
      Upload file ke bucket S3
      """
      if object_name is None:
          object_name = local_file
      try:
          s3.upload_file(
              local_file,
              bucket_name,
              object_name
          )
          print(f"✅ (object_name) berhasil diupload")
      except Exception as e:
          print("❌ Upload gagal")
          print(e)

[62]: upload_file("contoh2.txt")
      ✅ contoh2.txt berhasil diupload

[63]: response = s3.list_objects_v2(Bucket=bucket_name)
```

• List File

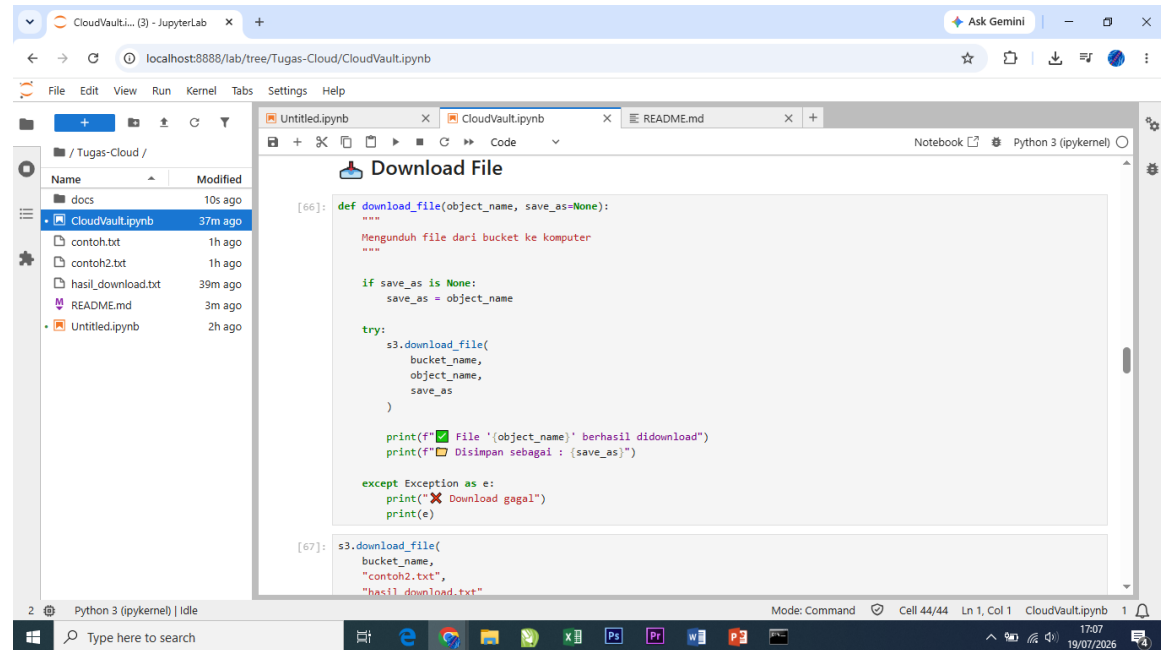


The screenshot shows a JupyterLab interface with a file browser on the left and a code editor on the right. The file browser displays the same directory structure as the previous screenshot. The code editor contains a Python function `list_files` that lists the contents of an S3 bucket. It uses the `s3.list_objects_v2` method to retrieve the bucket contents. The function checks if the response contains a 'Contents' key and prints the list of files. The notebook shows the execution of the function with `list_files()`, resulting in a list of files: `1. contoh2.txt (39 byte)`. The status bar at the bottom indicates the notebook is running on Python 3 (ipykernel) in Command mode.

```
[64]: def list_files():
      """
      Menampilkan semua file dalam bucket
      """
      response = s3.list_objects_v2(Bucket=bucket_name)
      if "Contents" not in response:
          print("📁 Bucket masih kosong.")
          return
      print("-" * 50)
      print("Daftar File dalam Bucket")
      print("-" * 50)
      for i, obj in enumerate(response["Contents"], start=1):
          print(f"[{i}]. {obj['Key']} ({obj['Size']} byte)")

[65]: list_files()
      =====
      Daftar File dalam Bucket
      =====
      1. contoh2.txt (39 byte)
```

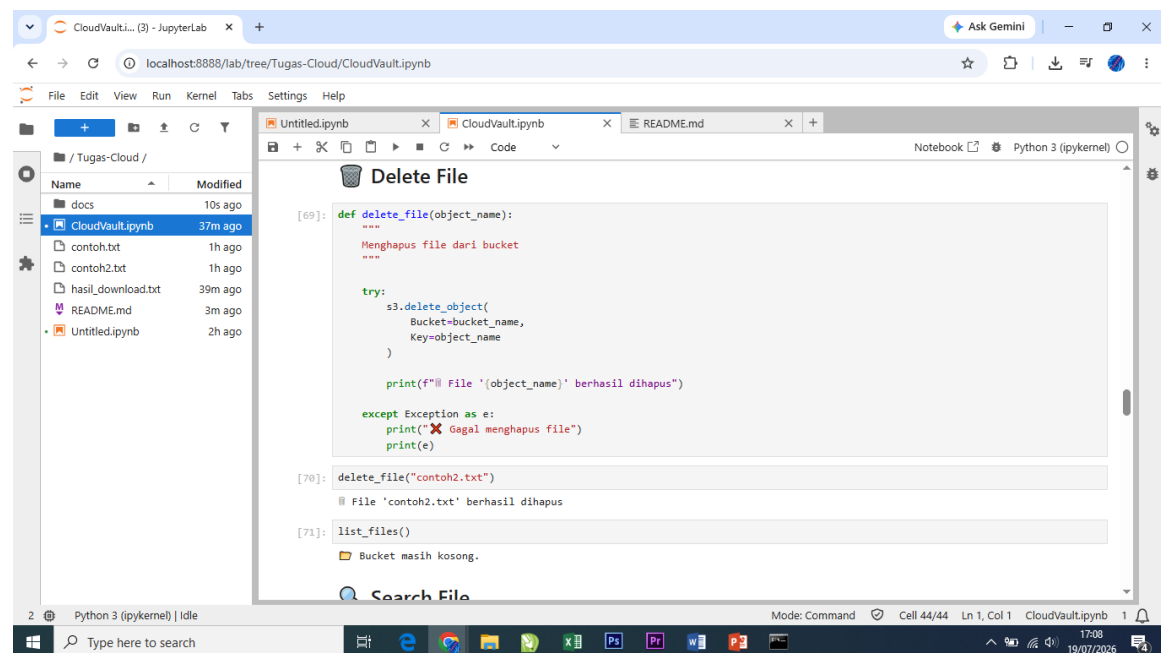
• Download File



The screenshot shows a JupyterLab environment with a file browser on the left and a code editor on the right. The file browser displays a directory structure for 'Tugas-Cloud /' with files like 'docs', 'CloudVault.ipynb', 'contoh.txt', 'contoh2.txt', 'hasil_download.txt', 'README.md', and 'Untitled.ipynb'. The code editor shows a Python notebook with a function 'download_file' and a call to it. The function uses 's3.download_file' to download a file from a bucket. The output shows the file 'contoh2.txt' being downloaded successfully and saved as 'hasil_download.txt'.

```
[66]: def download_file(object_name, save_as=None):  
    """  
    Mengunduh file dari bucket ke komputer  
    """  
  
    if save_as is None:  
        save_as = object_name  
  
    try:  
        s3.download_file(  
            bucket_name,  
            object_name,  
            save_as  
        )  
  
        print(f"✅ File '{object_name}' berhasil didownload")  
        print(f"📁 Disimpan sebagai : {save_as}")  
  
    except Exception as e:  
        print(f"❌ Download gagal")  
        print(e)  
  
[67]: s3.download_file(  
    bucket_name,  
    "contoh2.txt",  
    "hasil_download.txt"
```

• Delete File



The screenshot shows a JupyterLab environment with a file browser on the left and a code editor on the right. The file browser displays a directory structure for 'Tugas-Cloud /' with files like 'docs', 'CloudVault.ipynb', 'contoh.txt', 'contoh2.txt', 'hasil_download.txt', 'README.md', and 'Untitled.ipynb'. The code editor shows a Python notebook with a function 'delete_file' and a call to it. The function uses 's3.delete_object' to delete a file from a bucket. The output shows the file 'contoh2.txt' being deleted successfully. Below the function call, there is a call to 'list_files()' which outputs 'Bucket masih kosong.'.

```
[69]: def delete_file(object_name):  
    """  
    Menghapus file dari bucket  
    """  
  
    try:  
        s3.delete_object(  
            Bucket=bucket_name,  
            Key=object_name  
        )  
  
        print(f"✅ File '{object_name}' berhasil dihapus")  
  
    except Exception as e:  
        print(f"❌ Gagal menghapus file")  
        print(e)  
  
[70]: delete_file("contoh2.txt")  
      || File 'contoh2.txt' berhasil dihapus  
  
[71]: list_files()  
      || Bucket masih kosong.
```

• Search File

The screenshot shows a JupyterLab environment with a file browser on the left and a code editor on the right. The file browser displays a directory structure for 'Tugas-Cloud' containing files like 'docs', 'CloudVault.ipynb', 'contoh.txt', 'hasil_download.txt', 'README.md', and 'Untitled.ipynb'. The code editor is open to 'CloudVault.ipynb' and contains a Python function named 'search_file' that searches for a keyword in a bucket. The function uses 's3.list_objects_v2' to retrieve objects and checks if the keyword is in the object's key. If found, it appends the object to a list and prints the results. If not found, it prints a message indicating the file was not found.

```
[72]: def search_file(keyword):  
    """  
    Mencari file berdasarkan nama  
    """  
  
    response = s3.list_objects_v2(Bucket=bucket_name)  
  
    if "Contents" not in response:  
        print("❗ Bucket kosong")  
        return  
  
    hasil = []  
  
    for obj in response["Contents"]:  
        if keyword.lower() in obj["Key"].lower():  
            hasil.append(obj)  
  
    if len(hasil) == 0:  
        print("❌ File tidak ditemukan")  
    else:  
        print("-" * 45)  
        print("🔍 Hasil Pencarian")  
        print("-" * 45)  
  
        for i, obj in enumerate(hasil, start=1):  
            print(f"{i}. {obj['Key']} ({obj['Size']} byte)")
```

• Storage Statistics

The screenshot shows a JupyterLab environment with a file browser on the left and a code editor on the right. The file browser displays a directory structure for 'Tugas-Cloud' containing files like 'docs', 'CloudVault.ipynb', 'contoh.txt', 'hasil_download.txt', 'README.md', and 'Untitled.ipynb'. The code editor is open to 'CloudVault.ipynb' and contains a Python function named 'storage_statistics' that calculates the total number of files and the total size of objects in a bucket. The function uses 's3.list_objects_v2' to retrieve objects and calculates the total file count and total size. It then prints the results in a formatted output.

```
[78]: def storage_statistics():  
    response = s3.list_objects_v2(Bucket=bucket_name)  
  
    if "Contents" not in response:  
        print("-"*45)  
        print("❗ Storage Statistics")  
        print("-"*45)  
        print("Jumlah File : 0")  
        print("Total Size : 0 Byte")  
        return  
  
    total_file = len(response["Contents"])  
    total_size = sum(obj["Size"] for obj in response["Contents"])  
  
    print("-"*45)  
    print("📊 Storage Statistics")  
    print("-"*45)  
    print(f"Jumlah File : {total_file}")  
    print(f"Total Size : {total_size} Byte")  
    print(f"                : {total_size/1024:.2f} KB")  
    print(f"                : {total_size/1024/1024:.2f} MB")  
  
[79]: storage_statistics()  
  
=====  
📊 Storage Statistics  
=====  
Jumlah File : 2  
Total Size : 202 Byte
```

• Bucket Information

```
[80]: def bucket_information():
      response = s3.list_objects_v2(Bucket=bucket_name)

      print("="*45)
      print("# Bucket Information")
      print("="*45)

      print("Bucket Name :", bucket_name)

      if "Contents" not in response:
          print("Jumlah File : 0")
      else:
          print("Jumlah File :", len(response["Contents"]))

[81]: bucket_information()

=====
# Bucket Information
=====
Bucket Name : cloudvault-storage
Jumlah File : 2

Exit Program
```

• Exit Program

```
[82]: def exit_program():
      print("="*45)
      print("Terima kasih telah menggunakan CloudVault")
      print("Program selesai.")
      print("="*45)

[83]: exit_program()

=====
Terima kasih telah menggunakan CloudVault
Program selesai.
=====

Kesimpulan

CloudVault berhasil diimplementasikan menggunakan Python, Boto3, dan MiniStack sebagai emulator Amazon S3.

Fitur yang berhasil dibuat:

• Membuat bucket
```

BAB IX - Kesimpulan

CloudVault berhasil mengimplementasikan operasi dasar Object Storage menggunakan Amazon S3 API melalui MiniStack sehingga seluruh fitur dapat diuji tanpa menggunakan akun AWS.